

Informe técnico de la aplicación MediControl

Miguel Angel Andrade Arnez

Universidad Franz Tamayo

Facultad de Ingeniería

Carrera de Ingeniería de Sistemas

Sistemas Operativos Móviles y Embebidos

Ing. Christian Luis Lopez Bravo

6 de octubre de 2026

Informe técnico de la aplicación MediControl

Introducción, Objetivo y Entorno

MediControl digitaliza la gestión de pacientes y citas de una clínica. Esta entrega corresponde al Hito 3, tarea A3.2, avance del proyecto procesual. El objetivo es que el personal consulte y registre esa información en el dispositivo, sin depender de cuadernos.

El entorno es Flutter 3.44.7, Dart 3.12.2, Android SDK 36 y JDK 17 de Microsoft (Google, s.f.). El emulador es `MediControl_liviano`, un Pixel 3a con 2 GB de RAM. No se usa Android Studio. Desde `medicontrol/` se ejecuta:

```
flutter pub get
flutter emulators --launch MediControl_liviano
flutter run
```

Alcance por Módulo

Lo implementado en esta entrega cubre seis módulos. El resto queda como trabajo futuro. La sincronización con Firebase o una API todavía no está.

Cuadro 1

Módulos implementados en esta entrega y trabajo futuro

Módulo	Implementado en esta entrega	Trabajo futuro
Login	Correo y contraseña de demostración; la sesión queda en el dispositivo	Recuperar contraseña
Inicio	Accesos rápidos y la próxima cita	Recordatorios
Pacientes	Listar, buscar y dar de alta	—
Perfil	Datos, consultas pasadas y un tratamiento de ejemplo (texto fijo)	Historial clínico editable y CRUD de tratamientos
Agenda	Calendario del mes y citas del día elegido	—
Nueva cita	Paciente, fecha, hora y motivo; la cita debe ser futura	—

El usuario de demostración es `miguel@clinicademo.com` con la contraseña `demo1234`.

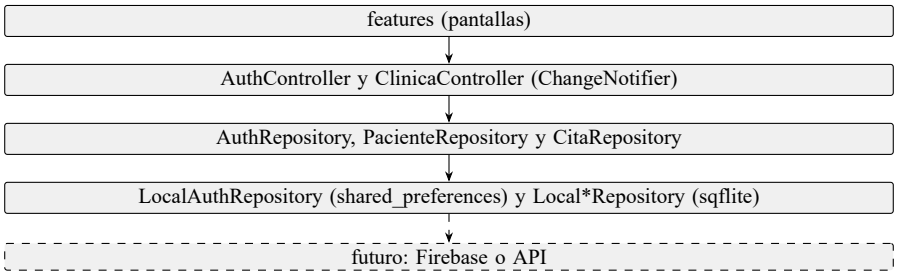
Arquitectura

Las pantallas no hablan con SQLite. La carpeta `features/` usa `AuthController` y `ClinicaController`, ambos `ChangeNotifier` (Flutter, s.f.; provider, s.f.). Esos controladores dependen de `AuthRepository`, `PacienteRepository` y `CitaRepository`.

Hoy las implementaciones son locales: `LocalAuthRepository` guarda la sesión con `shared_preferences` y `LocalPacienteRepository` junto con `LocalCitaRepository` usan sqflite (shared_preferences, s.f.; sqflite, s.f.). La línea punteada de la Figura 1 reserva el mismo contrato para un backend remoto. `lib/` reúne 35 archivos Dart y cerca de 2 400 líneas. La sección Estructura de carpetas dice qué hay en cada ruta.

Figura 1

Capas de MediControl. La línea punteada es trabajo futuro



Estructura de Carpetas

La Tabla 2 recorre las rutas reales. Salvo `figuras/` y el informe APA anterior, las rutas son relativas a `medicontrol/`. `aplicacion-medicontrol.tex` y su PDF describen la aplicación prevista; no son el código. Las cuatro últimas filas las generan Flutter, la compilación, el IDE o Gradle: no forman parte del código que se mantiene a mano.

Cuadro 2

Carpetas del repositorio y del paquete MediControl

Carpeta	Contenido	Para qué sirve
`figuras/`	`logo-unifranz.png`, `interfaz-medicontrol.png`	Escudo e interfaz del informe anterior
`aplicacion-medicontrol.tex` y `.pdf`	Informe APA previo	Describe la idea; no es la aplicación
`android/`	`app/` (manifiesto, `MainActivity.kt`, `res/`), `gradle.properties`, wrapper	Proyecto Gradle que empaqueta la app
`lib/`	`main.dart`, `app.dart`, `core/`, `data/`, `features/`, `widgets/`	Código Dart que se mantiene a mano
`lib/core/`	`consultas.dart`, `validators.dart`	Reglas de citas y validación, sin pantallas
`lib/core/format/`	`fechas.dart`	Comparar días y formatear fechas
`lib/core/router/`	`app_router.dart`	Rutas, shell y redirect de sesión
`lib/core/theme/`	`app_theme.dart`	Colores y tema visual
`lib/data/`	`models/`, `local/`, `repositories/`	Entidades, base local y contratos
`lib/data/models/`	`paciente.dart`, `cita.dart`, `usuario.dart`	Entidades en memoria
`lib/data/local/`	`app_database.dart`, `semilla.dart`, `session_store.dart`	SQLite, datos de demo y claves de sesión
`lib/data/repositories/`	Contrato y clase local de auth, paciente y cita (6 archivos)	Acceso local y enchufe para un backend

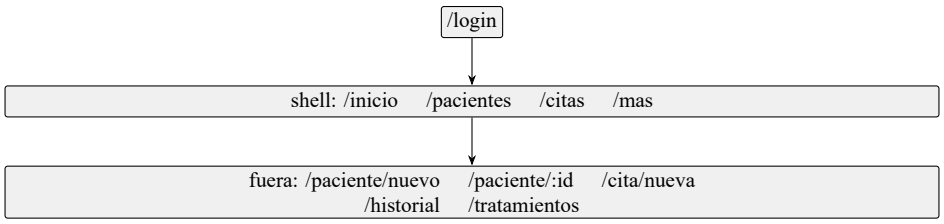
Carpeta	Contenido	Para qué sirve
`lib/features/`	auth, home, clinica, pacientes, perfil, agenda, citas, mas, proximamente	Una carpeta por módulo de la interfaz
`lib/features/auth/`	`login_screen.dart`, `auth_controller.dart`	Formulario de acceso y estado de sesión
`lib/features/home/`	`home_screen.dart`	Inicio, accesos y próxima cita
`lib/features/clinica/`	`clinica_controller.dart`	Catálogo, lista filtrada y agenda
`lib/features/pacientes/`	`pacientes_screen.dart`, `paciente_form_screen.dart`	Listado, búsqueda y alta
`lib/features/perfil/`	`perfil_screen.dart`, `tratamiento_fijo.dart`	Ficha, citas pasadas y texto fijo
`lib/features/agenda/`	`agenda_screen.dart`	Calendario mensual y citas del día
`lib/features/citas/`	`cita_form_screen.dart`	Alta de una cita futura
`lib/features/mas/`	`mas_screen.dart`	Usuario activo y cierre de sesión
`lib/features/proximamente/`	`proximamente_screen.dart`	Aviso de historial y de tratamientos
`lib/widgets/`	`app_shell.dart`, `cita_tile.dart`, `paciente_tile.dart`, `tarjeta.dart`	Barra inferior y piezas reutilizables
`test/`	`validadores_test.dart`, `modelos_test.dart`, `clinica_test.dart`, `widget_test.dart`	Pruebas automáticas del paquete
`test/soporte/`	`fakes.dart`	Sesión y repositorios en memoria
`capturas/`	`01-login.png` a `05-agenda.png`	Cinco pantallas del emulador
`lib/main.dart`	función `main`	Abre la base, carga la sesión y arranca
`lib/app.dart`	`MediControlApp`	Provider, idioma español y router
`pubspec.yaml`	Dependencias y SDK `^3.12.2`	Manifiesto del paquete
`pubspec.lock`	Versiones resueltas	Fija lo que se compiló
`analysis_options.yaml`	`include` de flutter_lints	Reglas de `flutteranalyze`
`README.md`	Requisitos, usuario demo y comandos	Cómo correr el paquete
`.gitignore`	Ignora `.dart_tool/`, `build/` y `.idea/`	Excluye generados del paquete
`.dart_tool/`	`package_config.json`, `package_graph.json`, `flutter_build/`	La genera Flutter; no se mantiene a mano
`build/`	`app/`, `.cxx/`, `reports/`	La genera la compilación; no se mantiene a mano
`.idea/`	`modules.xml`, `workspace.xml`, `runConfigurations/`	La genera el IDE; no se mantiene a mano
`android/.gradle/`	Caché `9.1.0/` y `buildOutputCleanup/`	La genera Gradle; no se mantiene a mano

Navegación y Modelo de Datos

`go_router` abre en `/inicio` y el redirect manda a `/login` si no hay sesión (go_router, s.f.). La barra inferior es un `StatefulShellRoute` con cuatro ramas: `/inicio`, `/pacientes`, `/citas` y `/mas`. Fuera del shell quedan `/paciente/nuevo`, `/paciente/:id`, `/cita/nueva`, `/historial` y `/tratamientos`. Historial y tratamientos muestran `ProximamenteScreen`.

Figura 2

Rutas: login, cuatro ramas del shell y pantallas fuera del shell

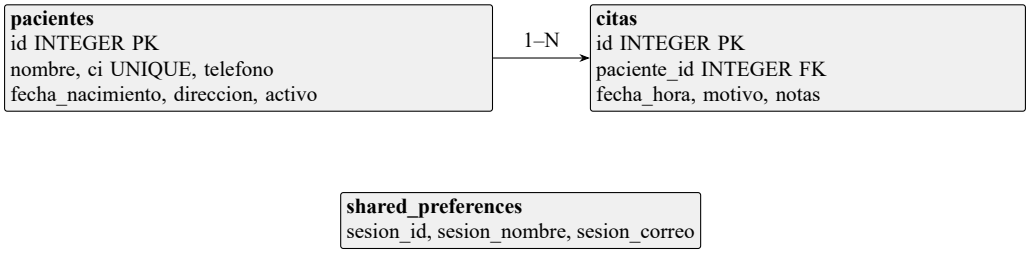


Cada paciente tiene varias citas. La clave foránea es `paciente_id`. La sesión no está en

SQLite: `PreferenciasSesion` escribe `sesion_id`, `sesion_nombre` y `sesion_correo`. Si `pacientes` está vacía, `AppDatabase.abrir` llama a `crearSemilla(DateTime.now())`. Entran Juan Pérez, María López, Carlos Andrade, Ana Torres y Luis Fernández, con citas de hoy, de mañana, dentro de tres días y varias pasadas (entre ellas -6 y -14 días).

Figura 3

Pacientes, citas y las tres claves de sesión



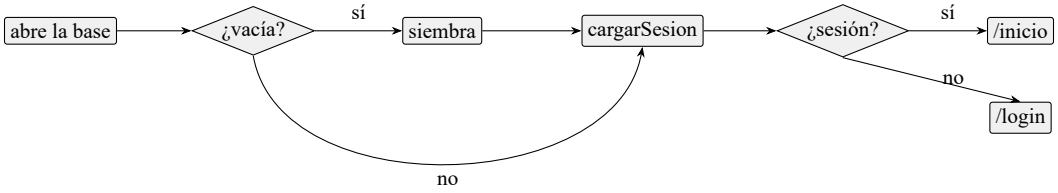
Lógica de Programación

Arranque y Sesión

`main` inicializa el idioma `es_ES`, abre la base (y siembra si está vacía), crea el controlador de autenticación y llama a `cargarSesion`. Esa lectura de tres claves es el mismo uso de SharedPreferences del Hito 3: si existen, el redirect deja pasar a `/inicio`; si no, devuelve `/login`. `refreshListenable` vuelve a evaluar el redirect cuando la sesión cambia.

Figura 4

Arranque: si la base está vacía se siembra y, si hay sesión, se abre el inicio




```
      return enLogin ? null : '/login';
    }
    if (enLogin) {
      return '/inicio';
```

Autenticación Local

`LocalAuthRepository` recorta el correo, lo pasa a minúsculas y lo compara con `CredencialesDemo`. Si no coincide, lanza `CredencialesInvalidas` y la pantalla de login

muestra el error. La contraseña va en texto plano a propósito, solo en memoria: la sesión guardada no la incluye.

Reglas de Citas

`proximaCita`, `citasEnDia` e `historialPaciente`, en `consultas.dart`, son funciones puras: no abren la base, y por eso se prueban con listas. `proximaCita` devuelve la cita con `fechaHora` más temprana que no sea anterior a ahora. `citasEnDia` filtra con `mismoDia` y ordena por hora. `historialPaciente` deja las citas pasadas de ese paciente, de la más reciente a la más antigua.

```
    if (cita.fechaHora.isBefore(ahora)) {
      continue;
    }
    if (proxima == null || cita.fechaHora.isBefore(proxima.fechaHora)) {
```

Búsqueda de Pacientes

El listado usa `LIKE` con `ESCAPE` sobre el nombre o el CI. `_like` escapa la barra, el porcentaje y el guion bajo para que el texto escrito no actúe como comodín. `ClinicaController` guarda el catálogo completo en `catalogo` y la lista filtrada en `lista`: el formulario de citas sigue viendo a todos.

```
            where: "nombre LIKE ? ESCAPE '\\' OR ci LIKE ? ESCAPE '\\'",
            whereArgs: [_like(texto), _like(texto)],
      .replaceAll(r'\', r'\\')
      .replaceAll('%', r'\%')
      .replaceAll('_', r'\_');
  return '%$limpio%';
```

Validación

Cuadro 3

Reglas de los formularios

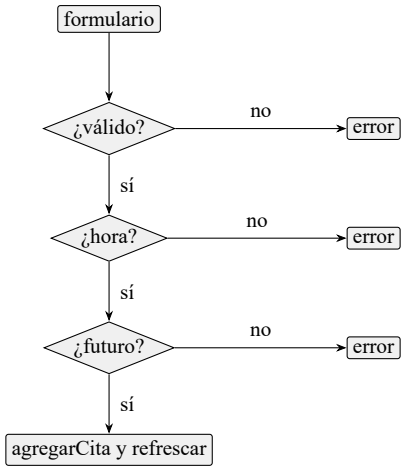
Campo	Regla	Mensaje
Correo	Vacío o sin formato	Ingresa el correo electrónico / El correo no tiene un formato válido
Contraseña	Al menos 6 caracteres	La contraseña debe tener al menos 6 caracteres
CI	5 a 10 dígitos y único	El CI debe tener entre 5 y 10 dígitos. Si ya existe: Ya existe un paciente con ese CI (`CiDuplicado`)
Teléfono	7 a 12 dígitos	El teléfono debe tener entre 7 y 12 dígitos

Guardar una Cita

El formulario tiene que ser válido, con fecha y hora elegidas. Esas partes se arman en un `DateTime`. Si no queda en el futuro, no se guarda. Si pasa, `agregarCita` inserta y `refrescar` vuelve a leer la base. Agenda e Inicio observan al controlador y se actualizan solos.

Figura 5

Comprobaciones al guardar una cita y camino de error




```
if (!fechaHora.isAfter(DateTime.now())) {
  setState(() => _error = 'La cita debe quedar en el futuro');
  return;
}
```

Ciclo con Provider

La pantalla llama al controlador con `context.read`. El controlador usa el repositorio, la base cambia y `notifyListeners` avisa. Quien hizo `context.watch` se redibuja. Por eso una cita nueva aparece en la agenda y en el inicio sin código extra entre esas pantallas.

Figura 6

Ciclo de estado: la pantalla pide, la base cambia y watch redibuja



Semilla Relativa

`crearSemilla(ahora)` calcula cada cita con `dia(offset,hora,minuto)` respecto del día de instalación. Así la agenda y la próxima cita tienen contenido el día en que se instala la aplicación.

Librerías y Configuración

Las versiones son las resueltas en `pubspec.lock` a partir de `pubspec.yaml`. `intl` quedó fijo en 0.20.2 porque lo exige `flutter_localizations`; por eso `table_calendar` es 3.2.1 y no 3.3.0 (intl, s.f.; table_calendar, s.f.).

Cuadro 4

Librerías directas y para qué se usan

Paquete	Versión	Uso
provider	6.1.5+1	Publica los dos controladores
go_router	17.5.0	Rutas, shell y redirect
shared_preferences	2.5.6	Claves de sesión
sqflite	2.4.4+1	Base `medicontrol.db`
path	1.9.1	Ruta del archivo de la base
intl	0.20.2	Datos de fecha en español
table_calendar	3.2.1	Mes de la agenda
flutter_localizations	SDK	Material en español
flutter_lints	6.0.0	Reglas de `flutteranalyze`

El `applicationId` es `com.unifranz.medicontrol` (Android Developers, s.f.). El módulo compila con JDK 17 (`sourceCompatibility`, `targetCompatibility` y `jvmTarget`). En `gradle.properties`, `-Xmx1536m` evita que Gradle pida 8 GB en un equipo de unos 14 GB con el emulador abierto (Gradle, s.f.). `kotlin.incremental=false` corta el fallo *different roots*: el proyecto está en D: y la caché en C:. `org.gradle.parallel=false` impide varios procesos Java a la vez. `org.gradle.workers.max=2` limita las tareas paralelas. `kotlin.daemon.jvmargs=-Xmx512m` acota el daemon de Kotlin. Esas cuatro claves y el tope de memoria están en el mismo archivo.

Pruebas y Evidencia

Hay 13 pruebas: tres en `validadores_test.dart`, cuatro en `modelos_test.dart`, dos en `clinica_test.dart` y cuatro en `widget_test.dart`. Las 13 pasaron. `flutteranalyze` terminó sin incidencias (flutter_lints, s.f.; path, s.f.).

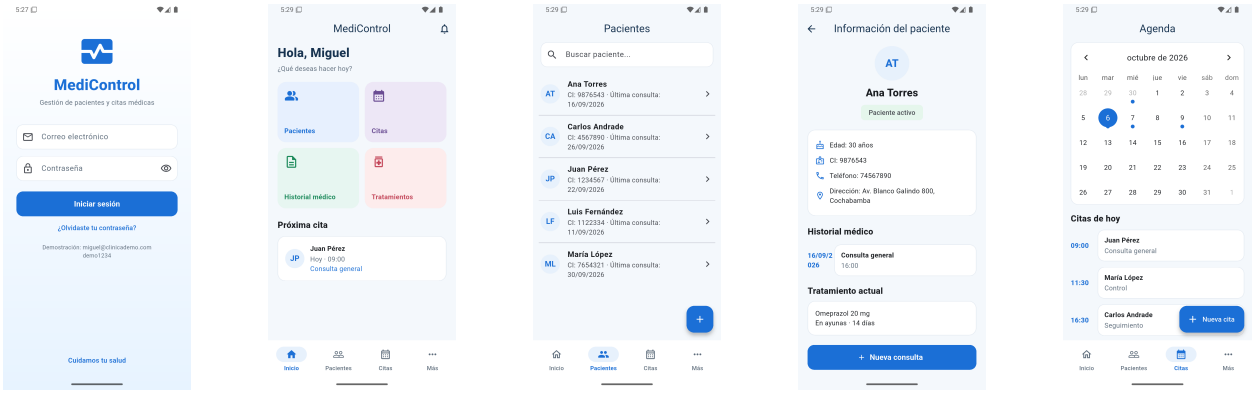
Cuadro 5

Archivos de prueba y qué cubren

Archivo	N	Qué cubre
validadores_test	3	Correo, contraseña de al menos 6 caracteres, CI y teléfono
modelos_test	4	Edad de Juan Pérez, mapa de la cita, semilla y próxima cita
clinica_test	2	La sesión reaparece al iniciar; buscar no vacía el catálogo
widget_test	4	Login vacío, clave incorrecta, olvido de contraseña e inicio con sesión

Figura 7

Emulador: inicio de sesión, inicio, pacientes, ficha de Ana Torres y agenda



Problemas, Trabajo Futuro y Conclusiones

El AVD Pixel 6 se congelaba: esta entrega usa el Pixel 3a de 2 GB y el disco de datos quedó en 6 GB, no en 10. El `intl` fijo, el fallo *different roots* y el tope de memoria de Gradle ya quedaron en la configuración.

Queda como trabajo futuro el historial clínico editable, el CRUD de tratamientos, los recordatorios, recuperar la contraseña y la nube. MediControl ya guarda pacientes y citas en el dispositivo, conserva la sesión y actualiza la agenda al registrar una cita. Separar pantallas, controladores y repositorios deja listo el lugar del almacenamiento remoto.

Referencias

Android Developers. (s.f.). *Set the application ID*. Consultado el 6 de octubre de 2026, desde https://developer.android.com/studio/build/application-id

Flutter. (s.f.). *ChangeNotifier class*. Consultado el 6 de octubre de 2026, desde https://api.flutter.dev/flutter/foundation/ChangeNotifier-class.html

flutter_lints. (s.f.). *flutter_lints package*. pub.dev. Consultado el 6 de octubre de 2026, desde https://pub.dev/packages/flutter_lints

Google. (s.f.). *Flutter documentation*. Consultado el 6 de octubre de 2026, desde https://docs.flutter.dev/

go_router. (s.f.). *go_router package*. pub.dev. Consultado el 6 de octubre de 2026, desde https://pub.dev/packages/go_router

Gradle. (s.f.). *Build environment*. Consultado el 6 de octubre de 2026, desde https://docs.gradle.org/current/userguide/build_environment.html

intl. (s.f.). *intl package*. pub.dev. Consultado el 6 de octubre de 2026, desde https://pub.dev/packages/intl

path. (s.f.). *path package*. pub.dev. Consultado el 6 de octubre de 2026, desde https://pub.dev/packages/path

provider. (s.f.). *provider package*. pub.dev. Consultado el 6 de octubre de 2026, desde https://pub.dev/packages/provider

shared_preferences. (s.f.). *shared_preferences package*. pub.dev. Consultado el 6 de octubre de 2026, desde https://pub.dev/packages/shared_preferences

sqflite. (s.f.). *sqflite package*. pub.dev. Consultado el 6 de octubre de 2026, desde https://pub.dev/packages/sqflite

table_calendar. (s.f.). *table_calendar package*. pub.dev. Consultado el 6 de octubre de 2026, desde https://pub.dev/packages/table_calendar